AEE. Bitácora III. Conexiones Empresariales
Taller Técnico: Operación Escudo

- **Módulo**: Sistemas Informáticos (0483)
- **Unidad Didáctica 6**: Seguridad y Gestión de Recursos en Red
- **Método de Entrega:** Repositorio GitHub (Documentación técnica en Markdown + Scripts de automatización)

Marco Evaluativo, Ponderación y Contexto Académico

Este taller constituye una actividad de evaluación sumativa de carácter práctico que computa directamente para la nota de la **Unidad Didáctica 6**. La intención es no solo demostrar destreza técnica frente a la consola de comandos, sino también vuestro rigor documental y vuestra capacidad de investigación autónoma utilizando fuentes de autoridad.

La verdad es que, en la industria actual del desarrollo y la administración de sistemas, un profesional que sabe ejecutar comandos pero no sabe documentarlos ni investigar el porqué de sus fallos, es un profesional incompleto. Por ello, esta práctica fusiona la administración pura con la cultura DevOps.

RESTRICCIÓN DE TIEMPO
TIMEBOXING

En el mundo real, los incidentes de seguridad no esperan a que terminéis vuestro café ni os dan semanas de plazo para investigar. Por esta razón, **solo disponéis de las 2 horas de duración de la clase de hoy** para investigar, ejecutar, documentar y hacer el *push* a GitHub de esta Bitácora. Vuestra capacidad de respuesta, análisis de problemas y organización bajo presión es exactamente lo que estamos evaluando. Lo que no esté subido al repositorio al finalizar la sesión, no será evaluado. Gestiona tu tiempo con inteligencia.

Resultados de Aprendizaje y Criterios de Evaluación

- **RA6**. Opera sistemas en red gestionando sus recursos e identificando las restricciones de seguridad existentes.
    - b) Identificación de derechos de usuario y aplicación de directivas de seguridad locales. (Se evaluará cómo limitas los accesos no deseados).

    - e) Evaluación crítica de la necesidad de proteger recursos críticos del sistema. (Entender por qué el ICMP puede ser un riesgo).
    - f) Instalación, configuración y evaluación de utilidades de seguridad perimetral (UFW) y auditoría.
- **RA7**. Elabora documentación valorando y utilizando aplicaciones informáticas de propósito general.
    - c) Realización de tareas de documentación técnica profesional mediante herramientas de control de versiones y lenguajes de marcado (Markdown/GitHub).
    - f) Uso proactivo de métodos de búsqueda de documentación técnica en repositorios científicos y técnicos (Dialnet, IEEE).

Ponderación de la Nota Final (0-10)

1. **Investigación y Referencias** (RA7). 20% (2 puntos). Se valorará la calidad de las fuentes, la profundidad del análisis y el rigor en el formato de cita.
2. **Configuración Técnica** (RA6). 40% (4 puntos). Verificación funcional de la seguridad perimetral y el correcto registro de los eventos del sistema.
3. **Automatización / Scripting** (RA6). 20% (Hasta 2 puntos). Calidad, robustez ante errores, comentarios y eficiencia del script desarrollado en Bash.
4. **Calidad de la Documentación en GitHub** (RA7). 20% (Hasta 2 puntos). Estructura semántica del repositorio, uso correcto de Markdown, evidencias visuales claras y un flujo de commits descriptivo.

1. El Escenario: "La Defensa en Profundidad y el rastro digital"

En la administración de sistemas moderna, la invisibilidad es una de las mejores defensas, pero nunca debe ser la única. Hablamos constantemente del concepto de **Defensa en Profundidad (Defense in Depth)**. Esto significa que no podemos confiar únicamente en una contraseña fuerte o en un cortafuegos; necesitamos múltiples capas superpuestas. Si un atacante logra traspasar la primera barrera, debe encontrarse con una segunda, y luego con una tercera. Y lo que es más importante: si logra encontrar la puerta, vuestro servidor debe ser capaz de registrar y alertar de cada uno de sus movimientos.

Y es que, para que os hagáis una idea, un servidor Linux recién expuesto en la nube (ya sea en AWS, Azure o GC) tarda, de media, menos de 5 minutos en recibir su primer escaneo de red automatizado. Existen redes enteras de bots rastreando Internet 24/7 en busca de puertos SSH abiertos y vulnerabilidades conocidas.

Ya no serás un usuario convencional; actuarás como **Administrador de Sistemas (SysAdmin)** en medio de un despliegue crítico y con el reloj en contra. El objetivo es claro: fortificar el sistema mediante técnicas de *Hardening*, levantar un muro de fuego

stateful (*Firewall*) y automatizar la vigilancia de los registros antes de que acabe vuestro turno.

Recordad el pilar fundamental de los sistemas basados en Unix: "**Todo es un fichero**". Desde la tarjeta de red virtual, pasando por la memoria RAM, hasta llegar a la configuración del firewall y los registros de seguridad. Entender esta arquitectura es la llave maestra para dominar cualquier servidor.

2. Fase de Investigación: Comprendiendo el corazón del sistema

Antes de realizar cambios a lo loco en un entorno de producción, un buen administrador debe investigar y fundamentar sus decisiones. La improvisación técnica suele derivar en fallos de seguridad críticos o en servidores inoperativos. Utilizad vuestro acceso a las bases de datos académicas [1] para dotar de base científica y metodológica a vuestro trabajo:

- **Reto de Investigación 1**. Anatomía de Syslog. Profundiza en el estándar Syslog, el sistema de mensajería vital de Linux. Investiga cómo el sistema clasifica los eventos cruzando dos variables: "Facilidad" (*Facility*, es decir, qué parte del sistema genera el log, como auth, cron o daemon) y "Prioridad" (*Severity*, desde *debug* hasta *emerg*).
    - Además, responde a esto: ¿Por qué es una negligencia grave que el archivo */var/log/auth.log* tenga permisos de lectura para usuarios no privilegiados?
    - ¿Qué información específica (como PIDs, nombres de usuario o direcciones IP) diferencia un intento fallido de conexión remota *SSH* de un simple fallo de contraseña de un usuario local frente a la pantalla?
- **Reto de Investigación 2**. Cumplimiento y Log Management. Busca en **Dialnet** o **Semantic Scholar** artículos sobre *Log Management* (Gestión centralizada de registros). A nivel empresarial y legal (piensa en el RGPD en España), ¿qué ventajas vitales ofrece enviar y custodiar los logs en un servidor externo seguro en lugar de mantenerlos dispersos e indefensos en la propia máquina que podría ser vulnerada?
- **Documentación obligatoria**. Crea un archivo **README.md** en la raíz de vuestro repositorio GitHub. La primera sección debe titularse "Fundamentos de Seguridad y Auditoría". En ella, debéis explicar todos estos conceptos analizados con vuestras propias palabras (nada de copiar y pegar). Por supuesto, debéis respaldar vuestras afirmaciones citando al menos dos fuentes reales y accesibles siguiendo estrictamente el manual de estilo IEEE [2].

3. Fase Práctica: *Hardening* y Vigilancia en la Trinchera

Esta fase debe ser documentada paso a paso. En una auditoría real, no basta con mostrar el resultado final; el "cómo se llegó ahí", las decisiones tomadas y los comandos ejecutados son la verdadera evidencia de vuestra competencia técnica.

Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting)

El archivo de registro de autenticación es vuestra "caja negra". Si alguien intenta forzar la entrada utilizando técnicas de *Fuerza Bruta* o *Ataques de Diccionario*, el rastro indeleble quedará grabado aquí.

1. **Generación de ataques controlados**. Desde vuestro equipo host (Windows) o desde una segunda máquina virtual, realiza al menos 5 intentos fallidos de conexión SSH contra vuestro servidor (ej: ejecutando *ssh administrador_falso@ip_del_servidor* y metiendo contraseñas inventadas).

```
joaquin@joaquin:~$ tail -f /var/log/auth.log
2026-04-17T11:36:55.632908+00:00 joaquin polkitd[8388]: Finished loading, compiling and executing 6 rules
2026-04-17T11:36:55.632942+00:00 joaquin polkitd[8388]: Reloading rules
2026-04-17T11:36:55.632952+00:00 joaquin polkitd[8388]: Collecting garbage unconditionally...
2026-04-17T11:36:55.633246+00:00 joaquin polkitd[8388]: Loading rules from directory /etc/polkit-1/rules.d
2026-04-17T11:36:55.633267+00:00 joaquin polkitd[8388]: Loading rules from directory /usr/share/polkit-1/rule
2026-04-17T11:36:55.633278+00:00 joaquin polkitd[8388]: Finished loading, compiling and executing 6 rules
2026-04-17T11:37:07.304606+00:00 joaquin polkitd[13588]: Loading rules from directory /etc/polkit-1/rules.d
2026-04-17T11:37:07.306808+00:00 joaquin polkitd[13588]: Loading rules from directory /usr/share/polkit-1/rul
2026-04-17T11:37:07.311957+00:00 joaquin polkitd[13588]: Finished loading, compiling and executing 6 rules
2026-04-17T11:37:07.316605+00:00 joaquin polkitd[13588]: Acquired the name org.freedesktop.PolicyKit1 on the

^C
joaquin@joaquin:~$ grep "Failed password" /var/log/auth.log
joaquin@joaquin:~$ _
```

```
joaquin@joaquin:~$ sudo apt install Fail2Ban
[sudo] password for joaquin:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
E: No se ha podido localizar el paquete Fail2Ban
joaquin@joaquin:~$ _
```

2. **Análisis Forense Manual**. Accede a */var/log/auth.log* utilizando herramientas de visualización rápida como *tail -n 50*, *less* o filtrando con *grep*. Debéis ser capaces de identificar y hacer una captura de pantalla donde se vea claramente: la dirección IP de origen del atacante, el puerto efímero utilizado y el mensaje literal de error del sistema (algo parecido a *Failed password for invalid user...*).

```
joaquin@joaquin:~$ sudo grep "Failed password" /var/log/auth.log | tail -n 50
2026-04-17T11:43:12.900508+00:00 joaquin sudo:  joaquin : TTY=tty6 ; PWD=/home/joaquin ; USER=root ; COMMAND=/usr/bin/grep 'Failed password' /var/log/auth.log
joaquin@joaquin:~$
```

3. **Automatización inteligente con Bash**. Como imaginaréis, un administrador no puede pasarse la mañana leyendo miles de líneas de texto plano. Debéis crear un *script* en *bash* en la ruta *scripts/check_intruders.sh* que automatice esta tarea. El script debe:
    - Filtrar exclusivamente los intentos fallidos de conexión de las últimas horas. *Pista*: Herramientas como *grep*, *awk* o *sed* serán vuestras mejores amigas para extraer columnas de texto.
    - Contar matemáticamente el número total de errores detectados.
    - Generar (o sobrescribir) un archivo de reporte limpio llamado **alertas.txt** que incluya un encabezado con la fecha y hora exacta de la generación del informe.
    - Requisito de calidad. El script no puede ser un bloque de código incomprensible. Debe estar **debidamente documentado** mediante **comentarios (#)** explicando qué hace exactamente cada variable, tubería (|) y comando utilizado.

```
joaquin@joaquin:~/scripts$ mkdir -p scripts
joaquin@joaquin:~/scripts$ cd scripts
joaquin@joaquin:~/scripts/scripts$
```

```
  GNU nano 7.2                                          check_introducers.sh *
#!/bin/bash

# ========================================
# NOMBRE: check_intruders.sh
# DESCRIPCION: Analiza  intentos fallidos de SSH y genera un reporte.
# ========================================

# Definicion de rutas de archivos
LOG_FILE="/var/log/auth.log"
REPORT_FILE="alertas.txt"

# 1. Crear/Sobrescribir el reporte con el encabezado y la fecha actual
echo "----------------------------------------" > $REPORT_FILE
echo "REPORT DE INSTRUSIONES - $(date '+%Y-%m-%d %H:%M:%S')" >> $REPORT_FILE
echo "----------------------------------------" >> $REPORT_FILE

# 2. Filtrar intentos fallidos y formatear la salida
# - grep: busca "Failed password2 en el log.
# -awk: imprime las columnas correspondientes a Fecha, Hora, Usuario, IP y Puerto.
echo -e "FECHA\tHORA\tUSUARIO\tIP_ORIGEN\tPUERTO" >> $REPORT_FILE

grep "Failed password" $LOG_FILE | awk ' {
     # Si el usuario es invalido, los datos se desplazan una posicion en la linea
     if ($9 == "invalid")
          print $1 " " $2 "\t" $9 "\t" $11 "\t" $13;
}' >> $REPORT_FILE

# 3. Contar mantematicamente el numero total de errores
# - grep -c: devuelve directamente el numero de lineas que coinciden.
TOTAL_ERRORES=$(grep -c "Failed password" $LOG_FILE)

# 4. Añadir el resumen final al reporte
echo -e "\n----------------------------------------" >> $REPORT_FILE
echo "TOTAL DE INTENTOS FALLIDOS DETECTADOS: $TOTAL_ERRORES" >> $REPORT_FILE
echo "----------------------------------------" >> $REPORT_FILE

# Mensaje de confirmacion por consola
echo "Analisis finalizado. Reporte generado en: $REPORT_FILE"

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-Q Previous
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-W Next
```

Reto B: Configuración del "Escudo" Perimetral con UFW

Bajo el capó de Ubuntu trabaja el potente motor de filtrado de paquetes *iptables* (o *nftables*). Sin embargo, nosotros utilizaremos **UFW (Uncomplicated Firewall)**, una interfaz amigable que nos facilita la vida. Vamos a aplicar la arquitectura más segura: una política de "Denegación por Defecto". Todo lo que no esté explícitamente invitado a entrar, se quedará fuera.

1. **Reglas de Sigilo** (*Drop* vs *Reject*). Configura el firewall para que el servidor ignore completamente las peticiones ICMP Echo Request (el famoso comando ping). Un servidor que hace drop (dejar caer el paquete sin responder) a los pings, se vuelve prácticamente invisible frente a los escaneos de red superficiales que buscan hosts vivos.
2. **Reglas de Servicio**. Por otro lado, si bloqueamos todo, nosotros tampoco podremos administrarlo. Permitid el acceso TCP exclusivamente al puerto 22 (SSH). Si, como buena práctica, habéis cambiado el puerto SSH por defecto a uno superior, aseguraos de abrir ese puerto específico.
3. **Activación y Evidencias**. Ejecuta *sudo ufw enable*. Posteriormente, verifica el estado y la carga real de las reglas con los comandos *sudo ufw status numbered* y *sudo ufw status verbose*.
4. **Prueba cruzada**. En vuestra documentación, debéis incluir capturas demostrando que desde la máquina anfitriona el *ping* da error de tiempo de espera, pero la conexión por cliente SSH (como Windows PowerShell o el Terminal) sí se establece correctamente.

```
joaquin@joaquin:~$ sudo ufw default deny incoming
[sudo] password for joaquin:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
joaquin@joaquin:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
joaquin@joaquin:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
joaquin@joaquin:~$ sudo ufw enable
Firewall is active and enabled on system startup
joaquin@joaquin:~$
```

4. Rúbrica de Evaluación Detallada y Feedback

Para que no haya sorpresas, la calificación de esta sesión se otorgará estrictamente en base al cumplimiento de los criterios descritos en la siguiente matriz. Además, durante los últimos 15 minutos de la clase, realizaremos una rápida inspección cruzada (Peer Review) donde comprobaréis la robustez del servidor de vuestro compañero de mesa.

Criterio	Excelente (9-10)	Bien (7-8)	Suficiente (5-6)	Insuficiente (0-4)
Investigación y Referencias (IEEE)	Cita fuentes académicas de gran impacto. Explica la arquitectura Syslog y las implicaciones legales del Log Management con una profundidad analítica sobresaliente. Formato IEEE perfecto.	Cita fuentes válidas pero la explicación peca de ser algo superficial o puramente descriptiva. Puede haber algún error menor de puntuación en el formato de cita.	La investigación es muy básica o divagante. Las citas provienen de foros o blogs sin autoridad, están incompletas o ignoran totalmente el estándar IEEE.	No existe investigación previa. Es un plagio directo de fuentes externas, “copia/pega” desde IA o se omiten las referencias bibliográficas.
Configuración Técnica UFW (RA6)	UFW activo con política de denegación, bloqueo total de ICMP y SSH permitido. Demuestra el éxito con pruebas cruzadas impecables y capturas de pantalla nítidas.	UFW activo y funcional, pero falta la regla específica de bloqueo de ICMP o las evidencias visuales de la prueba de verificación son confusas o incompletas.	El firewall funciona a nivel básico, pero no se han configurado las reglas de sigilo requeridas o el servicio SSH no responde adecuadamente.	Configuración errónea que deja el sistema vulnerable, o peor, bloquea el propio acceso del administrador perdiendo el control del servidor.
Automatización / Scripting Bash (RA6)	Script robusto, elegante e idempotente. Utiliza variables, procesa texto con agilidad y genera alertas.txt con un formato profesional. Documentación interna línea a línea excelente.	Script funcional que cumple el objetivo principal, pero carece de algunos comentarios explicativos o el formato de salida en el archivo de texto es algo desordenado o difícil de leer.	El script realiza la tarea a duras penas, presenta errores sintácticos menores, usa enfoques muy ineficientes o la lógica de filtrado es imprecisa.	El script no es ejecutable (falta de permisos chmod +x), tiene errores lógicos fatales o directamente no se incluye en la entrega.

Criterio	Excelente (9-10)	Bien (7-8)	Suficiente (5-6)	Insuficiente (0-4)
Calidad Documental (GitHub/MD)	Repositorio impecablemente estructurado (directorios lógicos como /scripts, /docs). README profesional con índices, fragmentos de código, capturas insertadas y narrativa clara.	README completo pero con una estructura visual mejorable. Las capturas están presentes pero carecen del pie de foto o la descripción técnica de lo que se está observando.	Repositorio desorganizado. Falta información relevante, uso pobre de la sintaxis Markdown o las evidencias visuales no demuestran fehacientemente el trabajo realizado.	El enlace proporcionado a GitHub es privado o erróneo. Repositorio vacío, subida de archivos sueltos sin README o documentación inexistente.

Referencias para la consulta

[1] W. Acosta Lugo, *Guía de uso de BBDD Académicas para Ciclos Formativos*, Sevilla, España: Departamento de Informática, 2026. Disponible en: https://drive.google.com/file/d/1Zg4LNDAs55OgEK1gmqbGx8NZyc1CdJjM/view?usp=sharing.

[2] IEEE, *IEEE Editorial Style Manual for Authors*, IEEE Periodicals, 2023. [En línea]. Disponible en: https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/

[3] Canonical Ltd., "Ubuntu Server documentation," Ubuntu Documentation, 2025. [En línea]. Disponible en: https://ubuntu.com/server/docs/explanation/intro-to/security/

[4] M. B. Alonso Alegre Díez, "Gestión de Logs," Trabajo Fin de Máster, Universidad Internacional de La Rioja (UNIR), 2016. [En línea]. Disponible en: https://reunir.unir.net/bitstream/handle/123456789/3618/ALONSO-ALEGRE%20DIEZ%2C%20MARIA%20BEGO%C3%91A.pdf

[5] S. McClure, J. Scambray y G. Kurtz, *Hacking Exposed: Network Security Secrets & Solutions*, 3ª ed. Nueva York, NY, EE. UU.: McGraw-Hill Education, 2001. Disponible en: https://github.com/jwx0539/hackingLibrary/blob/master/McGraw.Hill.Hacking.Exposed.Network.Security.Secrets.And.Solutions.3rd.Edition.Sep.2001.ISBN.0072193816.pdf